Roomify

Deployment

Proposed hosting, cost and steps (proposal only, not deployed)

Team Bootcamp2Roomify

Release v1.0.1 (live, demo mode)
24 October 2026
github.com/Bootcamp2Roomify/roomify_frontend · roomify_backend · roomify_cv

Deployment

Status (2026-10-26): live at https://roomify-rooms.vercel.app, in demo mode (`AI_DEMO_MODE=true`). Everything runs on free plans with no payment card: three Vercel Hobby projects and a Neon Free database.

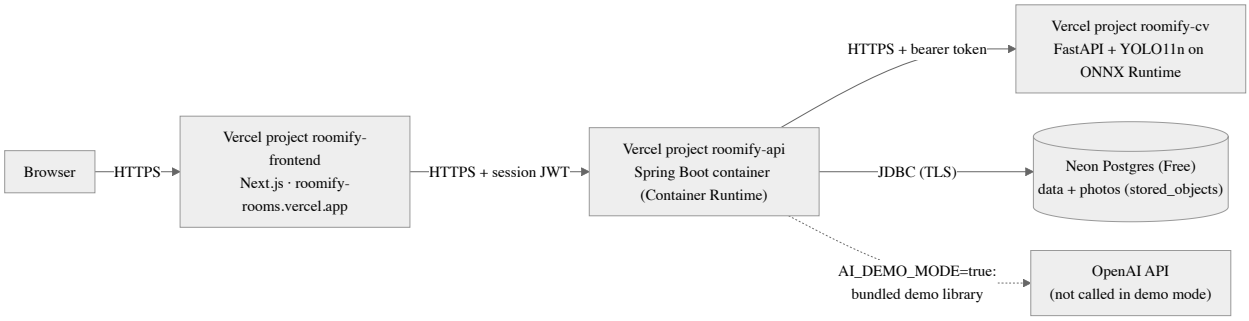


Figure 1. Roomify live deployment: three Vercel projects (Next.js frontend, Spring Boot container, FastAPI CV function) and Neon Postgres.

Piece	Where	URL
Frontend (Next.js)	Vercel project `roomify-frontend`	https://roomify-rooms.vercel.app
Backend (Spring Boot, Java 21)	Vercel project `roomify-api`, Container Runtime (beta)	https://roomify-api.vercel.app/api/health
CV service (FastAPI, YOLO11n)	Vercel project `roomify-cv`, Python Function on ONNX Runtime	https://roomify-cv.vercel.app/health
Database and photos	Neon Postgres Free, `us-east-2`	–

Why this shape

The owner asked for the backend on Vercel too, and for a host with no payment card. What was checked:

- **Vercel runs the JVM now.** Vercel Functions accept an OCI image built from a Dockerfile (Container Runtime, beta on all plans, Hobby included). It is not a long-running server: an instance serves requests, scales to zero after 5 idle minutes and must open its port within 28 s of a cold start. The backend fits that model: it keeps no state in memory, has no background jobs or schedulers (every request is handled synchronously), and stores photos in Postgres.
- **The CV service could not ship with torch.** The torch build uses about 510 MB of memory and far more than Vercel's 500 MB Python bundle limit. It now also runs on ONNX Runtime (roomify_cv#15): about 175 MB, same detections in practice (Evaluation Evidence §1).
- **Rejected:** Hugging Face Docker Spaces (a PRO subscription is required since 2026; creating the Space returned 402), Koyeb (card required since February 2026, per third-party reports), Render Free (512 MB per service, too small for the JVM plus YOLO in one container, and a split would expose the CV service on a second public URL anyway), Fly.io (payment method required).
- **No S3 or AWS key.** `STORAGE_TYPE=database` stores photos in a `stored_objects` table (migration V12). Vercel Blob has no Java SDK, and the database is already there. Neon Free holds 0.5 GB, plenty for judging; a demo-mode redesign adds about 60 kB.

- **The CV service is public, so it checks a token.** Vercel has no private network on Hobby. The backend sends `Authorization: Bearer $VISION_SERVICE_API_TOKEN`; the CV service rejects any other call to `/v1/analyze` or `/detect` with 401 (`CV_API_TOKEN`). `/health` stays open.

Cost

Piece	Plan	Monthly
Frontend, backend, CV service	Vercel Hobby (personal, non-commercial: fine for judging)	$0 within Hobby's included usage
Database and photos	Neon Free (0.5 GB, scales to zero after 5 min idle)	$0
OpenAI	not called while `AI_DEMO_MODE=true`	$0

Behaviour to expect

- **Cold starts.** After 5 idle minutes the backend scales to zero. The next request waits for the container: Spring starts in about 8 s on Vercel thanks to a class-data-sharing archive built into the image (13–18 s without it), and the very first request after a deploy took 22 s including the image pull. Neon adds up to a few seconds if the database was also asleep. Warm requests take under 1 s. The CV function cold-starts in about 1 s and analyses a photo in about 0.24 s. A scheduled GitHub Actions workflow (`.github/workflows/keep-warm.yml`, every 5 minutes) pings both services to keep them warm during judging; GitHub may delay scheduled runs, so a visitor can still hit a cold start, and Vercel may start a second instance (with its own cold start) when requests overlap.
- **Upload size.** Vercel Functions reject request bodies over 4.5 MB. The frontend re-encodes photos above 4 MB as JPEG (longest side 2560 px) before uploading (roomify_frontend#38).
- **Demo mode.** The plan, visual concept, suggestions and DIY guides come from the bundled demo library (fitted to the user's furniture decisions and budget; the concept is a stock photo for the room type), and every result is marked `source: "demo"` and labelled in the app. With `AI_DEMO_MODE=false` and a funded `OPENAI_API_KEY` the app calls OpenAI and falls back to the library if a call fails (`AI_DEMO_FALLBACK=true`). Real AI calls can take up to ~2 minutes; Hobby functions allow 300 s.

Steps

Prerequisites: a Vercel account (`vercel login`) and a Neon database. All three projects live in the same Vercel team.

1. **Database** — create a Neon project (Free) and note the host, database, user and password. The backend runs the Flyway migrations on first start; on an empty database that first start can miss Vercel's 28 s port deadline once (one 500), after which it is migrated and starts in about 8 s.
2. **CV service**

```
cd roomify_cv
deploy/vercel/build.sh /tmp/roomify-cv-vercel     # exports yolo11n.onnx, assembles
the bundle
cd /tmp/roomify-cv-vercel
vercel project add roomify-cv && vercel link --yes --project roomify-cv
printf %s yolo11n.onnx | vercel env add MODEL_PATH production
openssl rand -hex 32 | tr -d '\n' | vercel env add CV_API_TOKEN production   # keep
a copy for step 3
vercel deploy --prod
```

3. **Backend** (from the repo root; `.vercelignore` uploads only `Dockerfile.vercel`, `pom.xml` and `src`)

```
cd roomify_backend
vercel project add roomify-api && vercel link --yes --project roomify-api
vercel project update --framework container --yes   # pom.xml would otherwise hide
the Dockerfile
# production environment variables (vercel env add NAME production):
#   SPRING_DATASOURCE_URL=jdbc:postgresql://<neon-host>/<db>?sslmode=require
#   DB_USERNAME, DB_PASSWORD, JWT_SECRET=$(openssl rand -hex 32)
#   VISION_SERVICE_BASE_URL=https://roomify-cv.vercel.app
#   VISION_SERVICE_API_TOKEN=<CV_API_TOKEN from step 2>,
VISION_SERVICE_READ_TIMEOUT=30s
#   AI_DEMO_MODE=true, STORAGE_TYPE=database, PORT=8080,
SPRING_DATASOURCE_HIKARI_MAXIMUM_POOL_SIZE=3
vercel deploy --prod
curl https://roomify-api.vercel.app/api/health
```

   Neon requires TLS, so the JDBC URL is set whole with `sslmode=require` (it overrides `DB_HOST` / `DB_PORT` / `DB_NAME`). A small pool per instance keeps Neon's connection count low when Vercel runs several instances.
4. **Frontend** — project `roomify-frontend` (Next.js preset, Node 24.x), environment variable `ROOMIFY_API_URL=https://roomify-api.vercel.app`, then `vercel deploy --prod`. The public alias `roomify-rooms.vercel.app` was added as a project domain (`vercel domains add`) so it is not behind Vercel's deployment protection.
5. **Smoke test** — the live browser suite against the deployment, including a visual concept: `LIVE_BASE_URL=https://roomify-rooms.vercel.app LIVE_AI=1 LIVE_DEMO=1 npx playwright test -c playwright.live.config.ts`.
6. **After judging** — delete or pause the three Vercel projects; delete the Neon project.

Alternative: Fly.io (needs a payment method)

`roomify_backend/fly.toml` and `roomify_cv/fly.toml` remain for a host with always-on 1 GB machines, a private network for the CV service and S3 storage (`STORAGE_TYPE=s3` with a bucket-scoped IAM key). That setup costs about $13.40/month (two shared-cpu-1x machines at $6.70) and avoids cold starts.

Risks

- Container Runtime is a Vercel **beta**. If it changes, the Fly.io configuration is the fallback.
- `gpt-4o-mini` (default text model) is shut down by OpenAI on **2027-01-14**; switch to `gpt-6-luna` with `AI_TEMPERATURE=` and `AI_REASONING_EFFORT=low` before then (config only).

- Vercel Hobby is for non-commercial use; move to Pro ($20/month) for anything beyond judging.
- Before turning demo mode off on a public URL, set a monthly usage limit in the OpenAI dashboard.